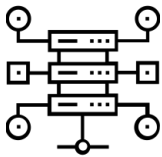


Assignment No: _5_
Date: 05 / 05 / 2026

Title:
(Title based on the application domain and the data structure you will be implementing)

Assignment Type of Submission:			
Group: 13	Yes/No	List all group members' details:	% Contribution Assignment Workload
We do the assignment separately this time.		Student Name: Jiawei Li Student ID: 25212461	100%



1. Problem Domain Description:

This project is about emergency evacuation route planning in a building. During an emergency, occupants may need to move from rooms or corridors to safe exits while avoiding routes that are smoky, crowded, blocked, or close to danger. A simple shortest physical route may not be the safest evacuation route, because a short corridor can still have high risk or congestion. The objective of the system is therefore to find a minimum-cost evacuation path rather than only the route with the fewest corridors.

Graphs are appropriate for this problem because a building can naturally be represented as a network. Rooms, corridors, stairwells, lobbies and exits are vertices. Walkable connections are edges. Each edge stores distance, risk and congestion, which are combined into a single evacuation cost. The implemented system compares BFS, which finds a fewest-edge route, with Dijkstra’s algorithm, which finds the route with the lowest total evacuation cost.

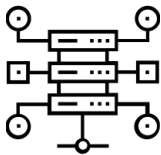
2. Theoretical Foundations of the Data Structure(s) utilised

Graph Element	Meaning in this Project
Vertex	Room, corridor, stairwell, lobby or exit
Edge	Walkable connection between two locations
Distance	Length or travel effort of a route segment
Risk	Danger level, such as smoke, heat or proximity to hazard
Congestion	Crowding level or expected delay
Evacuation cost	Combined edge weight used by Dijkstra’s algorithm

A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E connecting pairs of vertices. In this project the graph is undirected because movement is assumed possible in both directions along a corridor unless an edge is removed to simulate a blocked route. The graph is also weighted because each route segment has a cost. An adjacency-list representation is used because it stores only existing edges and is therefore space-efficient for sparse networks such as building layouts.

For each edge, the evacuation cost is calculated as follows:
 $\text{cost} = \text{distance} + 2 * \text{risk} + \text{congestion}$

Risk is weighted twice as heavily as distance because, in an evacuation scenario, safety is more important than taking the physically shortest route. This does not force the algorithm to always choose the safest-looking route; instead, it balances distance, risk and congestion into



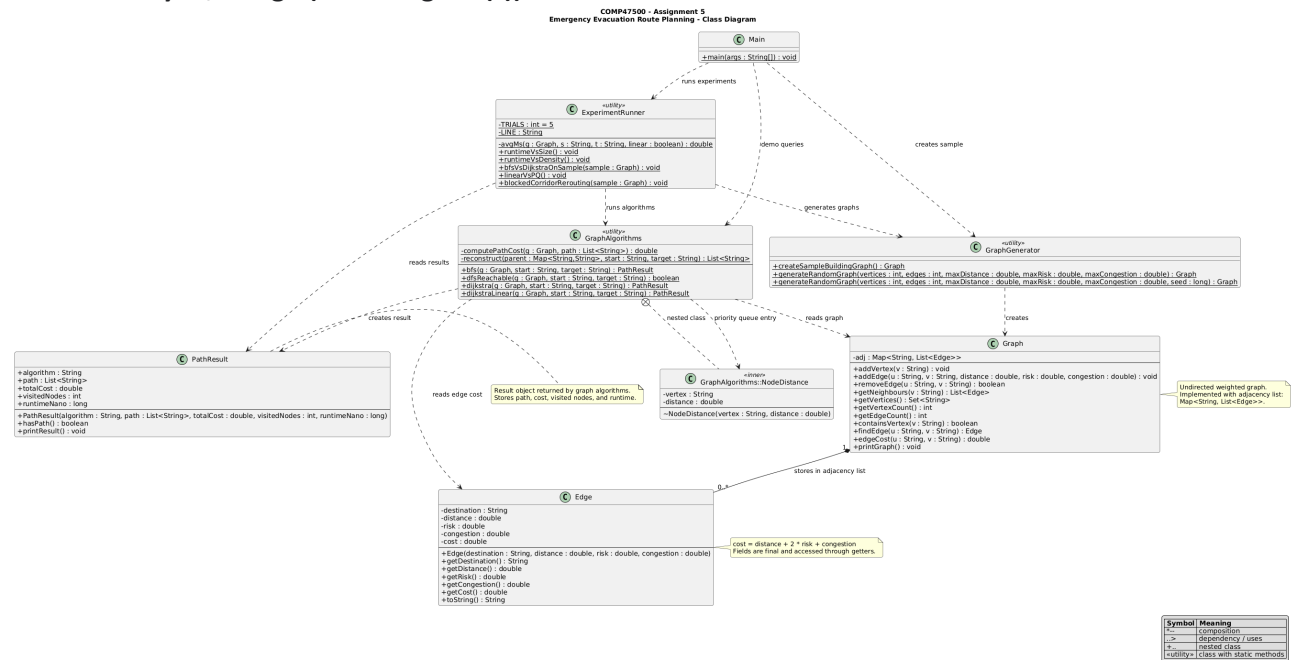
COMP47500 – Advanced Data Structures in Java

one cost value. Dijkstra's algorithm then chooses the path with the minimum total accumulated cost.

Algorithm	Purpose in System	Time Complexity	Space Complexity
BFS	Fewest-edge route in an unweighted view of the graph	$O(V + E)$	$O(V)$
DFS	Reachability checking between two locations	$O(V + E)$	$O(V)$
Dijkstra - linear scan	Weighted shortest path using linear minimum selection	$O(V^2 + E)$	$O(V)$
Dijkstra - PriorityQueue	Weighted shortest path using heap-based minimum selection	$O((V + E) \log V)$	$O(V + E)$

BFS and DFS are linear in the size of the graph because each vertex and edge is processed at most once. Linear-scan Dijkstra is less efficient because each iteration searches all unvisited vertices to find the current smallest distance. The PriorityQueue version improves practical efficiency by making the next-vertex selection heap-based, which is more suitable for large sparse graphs. This is why the PriorityQueue implementation is used as the main weighted routing algorithm.

3. Analysis/Design (UML Diagram(s))



4. Code Implementation (please my github account as – <https://github.com/jrgarga> - as a collaborator)

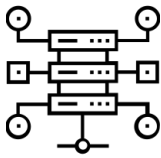
GitHub (link):

<https://github.com/17Esther/Emergency-Evacuation-Route-Planning>

Set of Experiments run and results:

File	Purpose
Edge.java	Defines immutable weighted edge objects and the evacuation cost formula.
Graph.java	Implements the custom undirected adjacency-list graph.
PathResult.java	Stores path, cost, visited-node count and runtime.
GraphAlgorithms.java	Implements BFS, DFS, Dijkstra with PriorityQueue and linear Dijkstra.
GraphGenerator.java	Creates sample and random graphs.
ExperimentRunner.java	Runs all experimental tests and prints tables.
Main.java	Demonstrates the system and runs all experiments.

All experiments were executed from the Main class after compiling the project from the terminal. Runtime was measured using System.nanoTime(). For Experiments 1, 2



and 4, each input setting used one warm-up run followed by five measured trials, and the average runtime in milliseconds is reported. Random graphs were generated with fixed seeds so the experiments are reproducible. The main measured metrics are graph size, edge count, average runtime, total evacuation cost, number of visited nodes and speedup.

The sample building graph is used as a controlled case study rather than a random performance dataset. Its purpose is to demonstrate algorithm behaviour: BFS optimises for the fewest corridors, while Dijkstra optimises for the lowest accumulated evacuation cost. The random graphs are used for scalability and efficiency experiments.

Experiment 1: Dijkstra runtime vs graph size

This experiment evaluates how Dijkstra's runtime changes as the number of vertices increases. The graph density was kept approximately stable by using $E = 3V$. Graph sizes ranged from 100 to 5000 vertices. For each graph, the algorithm searched from V0 to the final vertex. The PriorityQueue version was measured for all graph sizes, while the linear version was skipped at 5000 vertices to avoid unnecessary long runtime.

```
=== Experiment 1: Dijkstra runtime vs graph size (E = 3V, avg of 5 trials) ===
```

V	E	Avg PQ time (ms)	Avg Linear (ms)	PQ cost
100	300	0.147	0.603	41.976
500	1500	0.287	1.407	36.586
1000	3000	0.785	12.485	47.551
3000	9000	1.593	19.667	58.679
5000	15000	1.584	skipped	58.433

Analysis: The PriorityQueue Dijkstra runtime generally increased as the graph became larger, rising from 0.147ms at 100 vertices to around 1.58ms at 5000 vertices. This supports the expected complexity trend because larger graphs contain more vertices and edges to process. The 3000-to-5000 result is almost flat rather than sharply increasing, which is acceptable because Dijkstra stops once the target is reached and runtime is also affected by graph topology, JVM warm-up and garbage collection. The linear implementation became much slower as graph size increased, reaching 19.667ms at 3000 vertices. This confirms that repeatedly scanning all unvisited vertices is less efficient than using a heap-based PriorityQueue for sparse graphs.

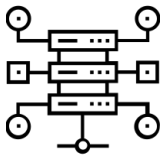
Experiment 2: Dijkstra runtime vs graph density

This experiment keeps the number of vertices fixed at 1000 and increases the number of edges from 1000 to 20000. This tests the effect of graph density on shortest-path performance. Each setting was averaged over five measured trials after one warm-up run.

```
=== Experiment 2: Dijkstra runtime vs density (V = 1000, avg of 5 trials) ===
```

V	E	Avg PQ time (ms)	Avg Linear (ms)	PQ cost
1000	1000	0.064	0.714	137.942
1000	3000	0.095	0.935	44.862
1000	5000	0.351	2.701	44.862
1000	10000	0.610	2.317	33.699
1000	20000	0.811	3.998	22.856

Analysis: The PriorityQueue runtime increased as the graph became denser, from 0.064ms with 1000 edges to 0.811ms with 20000 edges. This is expected because each



COMP47500 – Advanced Data Structures in Java

vertex has more neighbours, so the relaxation step has more edges to inspect. The path cost generally decreased as more edges were added, because a denser graph gives the algorithm more alternative routes and therefore a higher chance of finding a lower-cost path.

The linear version was slower than the PriorityQueue version for all density levels. Its runtime is not perfectly monotonic because the exact vertices settled before reaching the target can vary with graph structure, but the overall result still shows that the PriorityQueue approach is more efficient.

Experiment 3: BFS vs Dijkstra on the controlled sample building graph

This controlled experiment compares the route selected by BFS with the route selected by Dijkstra's algorithm on the manually designed building graph. The start location was RoomA and the target was Exit1. BFS ignores weights and finds the route with the fewest corridors. Dijkstra uses the evacuation cost of each edge and finds the lowest-cost route.

```

=== Experiment 3: BFS vs Dijkstra on sample building graph ===
From RoomA to Exit1:
---- BFS (fewest edges) ----
  Path      : RoomA -> Corr1 -> Stair1 -> Exit1
  Hops      : 3
  Total cost : 72.000
  Visited   : 7
  Runtime   : 0.013 ms (12625 ns)
---- Dijkstra (PQ) ----
  Path      : RoomA -> RoomB -> Corr2 -> Lobby -> Exit1
  Hops      : 4
  Total cost : 39.000
  Visited   : 9
  Runtime   : 0.063 ms (63417 ns)
Note: BFS picks the path with the FEWEST corridors (often the smoky/risky one).
      Dijkstra picks the path with the LOWEST evacuation cost (safer).

```

Analysis: BFS found the route with fewer hops, but that route had a much higher evacuation cost of 72.000 because it used smoky, risky and crowded edges. Dijkstra chose a longer route with four hops, but its total evacuation cost was only 39.000. This demonstrates the main theoretical idea of the project: in emergency evacuation, the shortest route by number of corridors is not always the safest or most suitable route.

This case study is not intended as a scalability test. Instead, it validates the correctness and usefulness of the weighted graph model by showing that different algorithms optimise different criteria.

Experiment 4: Linear-scan Dijkstra vs PriorityQueue Dijkstra

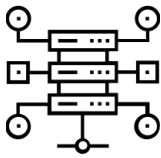
This experiment measures the practical efficiency gained by replacing linear minimum-distance selection with a PriorityQueue. Both algorithms return the same type of weighted shortest path, but they differ in how the next closest vertex is selected. Graph sizes ranged from 200 to 2000 vertices, with $E = 3V$.

```

=== Experiment 4: Linear-scan Dijkstra vs PriorityQueue Dijkstra (avg of 5 trials) ===
+-----+-----+-----+-----+-----+
| V      | E      | Avg PQ time (ms) | Avg Linear (ms) | Speedup x |
+-----+-----+-----+-----+-----+
| 200    | 600    | 0.080            | 0.125            | 1.55       |
| 500    | 1500   | 0.057            | 0.169            | 2.95       |
| 1000   | 3000   | 0.079            | 0.394            | 4.97       |
| 2000   | 6000   | 0.959            | 10.858           | 11.32      |
+-----+-----+-----+-----+-----+

```

Analysis: The PriorityQueue implementation was faster for every tested graph size. The speedup increased from 1.55x at 200 vertices to 11.32x at 2000 vertices, showing that the heap-based implementation becomes more valuable as input size grows. This



COMP47500 – Advanced Data Structures in Java

result supports the theoretical difference between linear-scan Dijkstra, which repeatedly searches through vertices, and PriorityQueue Dijkstra, which performs more efficient minimum selection.

Experiment 5: Blocked corridor rerouting

This experiment tests whether the graph can adapt when a corridor becomes unavailable. The system first runs Dijkstra from RoomA to Exit1. Then the edge RoomA <-> RoomB, which is part of the best route, is removed to simulate a blocked corridor. Dijkstra is run again on the modified graph.

```
=== Experiment 5: Blocked corridor rerouting ===
Before blocking any corridor:
---- Dijkstra (PQ) ----
Path      : RoomA -> RoomB -> Corr2 -> Lobby -> Exit1
Hops       : 4
Total cost : 39.000
Visited    : 9
Runtime    : 0.015 ms (15125 ns)

Blocking corridor: RoomA <-> RoomB (d=6.00, r=1.00, c=1.00)
After blocking:
---- Dijkstra (PQ) ----
Path      : RoomA -> Corr1 -> Stair1 -> Exit1
Hops       : 3
Total cost : 72.000
Visited    : 4
Runtime    : 0.002 ms (1833 ns)
(corridor restored to original weights)
```

Analysis: Before the blockage, Dijkstra selected the safer low-cost route through RoomB, Corr2 and Lobby. After the RoomA to RoomB corridor was removed, that route was no longer available, so the algorithm recalculated and found an alternative path through Corr1 and Stair1. The new path had a higher cost of 72.000, showing that the system can reroute when the graph structure changes, although the replacement route may be less safe or more expensive.

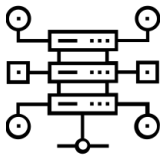
Comments:

A key design decision was to use an adjacency-list representation rather than an adjacency matrix. This is more suitable for the evacuation domain because real building layouts are usually sparse: each room or corridor connects to only a small number of neighbouring locations. With an adjacency list, the graph stores only existing edges, reducing unnecessary memory usage and allowing algorithms to iterate directly over relevant neighbours.

Another important decision was to use a combined edge weight instead of modelling distance, risk and congestion separately during the search. The formula $\text{cost} = \text{distance} + 2 * \text{risk} + \text{congestion}$ keeps the implementation simple while still expressing the main evacuation trade-off: safety should be prioritised, but distance and congestion still matter. This also allows Dijkstra's algorithm to be applied directly to a single weighted graph.

The project deliberately includes both BFS and Dijkstra because they demonstrate different algorithmic assumptions. BFS is useful as a baseline for the fewest-corridor route, but it ignores edge weights and can therefore choose an unsafe route. Dijkstra is more appropriate for this problem because the best evacuation path depends on accumulated cost rather than only the number of edges. The linear Dijkstra implementation was included to show how the practical efficiency of the same algorithm can be improved through a PriorityQueue.

The main implementation challenge was designing a sample building graph that clearly demonstrates the difference between unweighted and weighted routing. The graph had to include a short but risky route and a longer but safer route so that BFS and Dijkstra would produce different meaningful outputs. Another challenge was making timing results more reliable. To reduce noise, the final version uses fixed random seeds, one warm-up run and five measured trials per input setting.



COMP47500 – Advanced Data Structures in Java

The main limitation is that the evacuation cost formula is simplified. In a real system, distance, risk and congestion would need to come from sensors, building maps or emergency management systems, and the weights may need to change dynamically during an emergency. The random graphs used in the performance experiments also do not fully represent real building floor plans. Future work could add dynamic risk updates, multiple exits, comparison with A* search, more realistic map data, and visualisation of the building layout and selected evacuation route.

5. Video of the Implementation running

[Zoom \(link & password\):](#)

https://ucd-ie.zoom.us/rec/share/5sBsQzAulY3lzag_cdW1nRYWJhTbulFCX4z3u2L2zy7JT7-cBD_NU1x9xGeGgOl.uq2_P9e6_cC_agtY

Passcode: i!XN&73*

Please save as pdf and submit on Brightspace
Students belonging to the same group please submit the same file .